

Módulo: Programación.

Unidad de Trabajo N°6: Ficheros y Acceso a BBDD.

Actividad de Desarrollo 2: Sentencias SQL desde Java.

1) Objetivo general.

Integrar los conceptos de conectores y Bases de Datos y la utilización de sentencias SQL desde programas desarrollados en Java.

2) Metodología.

Cada alumno realizará de forma individual las cuestiones planteadas en la actividad.

3) Descripción.

Vamos a conectarnos a las Base de Datos utilizando JDBC de forma manual sin Proyecto Maven o Gradle.

1. Introducción

Fuente de la Introducción a JDBC Acceso a datos – Desarrollo de Aplicaciones Multiplataforma – Autor : Inazio Claver) y elaboración propia.

Introducción a JDBC

JDBC (Java DataBase Connectivity) es la API estándar de acceso a base de datos desde Java.

Para conectarse a una base de datos concreta, es necesario su driver JDBC. El driver es un fichero JAR que se añade a la aplicación como cualquier otra librería (no necesita instalación adicional), y la mayoría de las bases de datos incorporan un driver JDBC.

ODBC (Open DataBase Connectivity) es un estándar de acceso a base de datos desarrollado por Microsoft. Sun (posteriormente adquirido por Oracle) ha desarrollado un driver que hace de puente entre JDBC y ODBC aunque no suele usarse

Los pasos para que una aplicación se comuniquen con una base de datos son y un ejemplo de código Java para conectarse a MySQL:

1.1. Carga del driver

```
Class.forName("com.mysql.jdbc.Driver");
```

Antes de poder conectarse a la base de datos es necesario cargar el driver JDBC, y solo hay que hacerlo una única vez al comienzo de la aplicación. El nombre del driver debe venir especificado en la documentación de la base de datos.

1.2. Establecer una conexión

```
Connection conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/sample",  
"root", "password");
```

Las bases de datos actúan como servidores y las aplicaciones como clientes que se comunican a través de la red.

Un objeto **Connection** representa una conexión física entre el cliente y el servidor. Para crear una conexión se usa la clase `DriverManager`, especificando la URL, el nombre y la contraseña de nuestra base de datos.

El formato de la URL debe especificarse en el manual de la base de datos. En el ejemplo es una muestra de como conectar a MySQL. Por supuesto, el nombre de usuario y la contraseña también dependen de la base de datos.

4)

1.3. Enviar consultas SQL de Selección y procesar el resultado

```
Statement stmt = conn.createStatement();  
ResultSet rs = stmt.executeQuery("SELECT titulo, precio FROM Libros WHERE  
precio > 2");
```

Una vez que tienes una conexión puedes ejecutar sentencias SQL.

Primero se crea el objeto **Statement** desde la conexión, y posteriormente se ejecuta la consulta y su resultado se devuelve como un **ResultSet**.

Acceso al conjunto de resultados:

```
while(rs.next()) {  
    String name = rs.getString("titulo");  
    float price = rs.getFloat("precio");  
    System.out.println(name + "\t" + price);  
}
```

El `ResultSet` es el objeto que representa el resultado.

No carga toda la información en memoria, internamente tiene un cursor en memoria que apunta a una fila concreta del resultado en la base de datos, por lo que hay que posicionar el cursor en cada fila y obtener toda la información de la misma.

1.4.Otras Consultas (DELETE, INSERT Y UPDATE).

Insertar datos en una tabla

Usando Statement: (en este ejemplo utiliza variables no datos fijos)

```
//En vez de poner la consulta directamente en el executeUpdate creamos un string
String query = "INSERT INTO autores VALUES("+i+", 'autor"+i+"', "+i+")";
sentencia.executeUpdate(query);
```

Usando PreparedStatement:

```
String query = "INSERT INTO autores VALUES(?,?,?)";
PreparedStatement psi=conexion.prepareStatement(query);
psi.setInt(1,12);
psi.setString(2,"Relano");
psi.setInt(3,69);
psi.executeUpdate();
```

Actualizar datos en una tabla

Usando Statement (en el ejemplo solicita al usuario el código del autor a modificar y la edad a introducir.):

```
Scanner tec = new Scanner(System.in);
System.out.println("Introduce el código de autor: ");
int autor=tec.nextInt();
System.out.println("Introduce la nueva edad del autor: ");
int newedad=tec.nextInt();
String query = "UPDATE autores SET edad="+newedad+" WHERE codautor="+autor;
int nborrados = sentencia.executeUpdate(query);
if (nborrados!=0) {
    System.out.println("Registro actualizado.");
} else {
    System.out.println("No se encontró el registro");
}
```

Usando PreparedStatement:

```
Scanner tec = new Scanner(System.in);
System.out.println("Introduce el código de autor: ");
int autor=tec.nextInt();
System.out.println("Introduce la nueva edad del autor: ");
int newedad=tec.nextInt();
String query = "UPDATE autores SET edad=? WHERE codautor=?";
PreparedStatement ps=conexion.prepareStatement(query);
ps.setInt(1,newedad);
ps.setInt(2,autor);
int nactualizados = ps.executeUpdate();
if (nactualizados!=0) {
    System.out.println("Registro actualizado.");
} else {
    System.out.println("No se encontró el registro");
}
```

Eliminar registros

Utilizando Statement: (en el ejemplo se solicita al usuario el código del autor a eliminar:

```
Scanner tec = new Scanner(System.in);
System.out.println("Introduce el código de autor: ");
int autor=tec.nextInt();
String query = "DELETE FROM autores WHERE codautor="+autor;
int nborrados = sentencia.executeUpdate(query);
if (nborrados!=0) {
    System.out.println("Registro borrado.");
} else {
    System.out.println("No se encontró el registro");
}
```

Usando PreparedStatement:

```
Scanner tec = new Scanner(System.in);
System.out.println("Introduce el código de autor: ");
int autor=tec.nextInt();
String query1 = "DELETE FROM autores WHERE codautor=?";
PreparedStatement ps=conexion.prepareStatement(query1);
ps.setInt(1,autor);
int neliminados = ps.executeUpdate();
if (neliminados!=0) {
    System.out.println("Registro eliminado.");
} else {
    System.out.println("No se encontró el registro");
}
```

1.5. Liberar los recursos al terminar

```
rs.close();  
stmt.close();  
conn.close();
```

Cuando se termina de usar una Connection, un Statement o un ResultSet es necesario liberar los recursos que necesitan.

Puesto que la información de un ResultSet no se carga en memoria, existen conexiones de red abiertas.

Métodos close():

ResultSet.close() – Libera los recursos del ResultSet. Se cierran automáticamente al cerrar el Statement que lo creó o al reejecutar el Statement.

Statement.close() – Libera los recursos del Statement.

Connection.close() – Finaliza la conexión con la base de datos.

1.6. Manejar los errores que se puedan producir

Hay que gestionar los errores apropiadamente, ya que se pueden producir excepciones **ClassNotFoundException** si no se encuentra el driver, o excepciones **SQLException** al interactuar con la base de datos.

Por ejemplo:

SQL mal formado

Conexión de red rota

Problemas de integridad al insertar datos (claves duplicadas)

throws ClassNotFoundException, SQLException

Lo que hacemos con la sintaxis anterior es dejar que el error lo gestionen en un nivel superior, pero si lo queremos controlar nosotros mismo tenemos que utilizar

```
try {  
    } catch () {  
    }
```

2. Programa/s en Java.

Puedes desarrollar las clases que consideres. Pero intenta que sea lo más modular posible. Utiliza el SGBD MySQL:

2. Crea la siguiente base de datos en MariaDB (o MySQL) con la siguientes tablas:

Departamentos	Empleados
• Deptno numérico, clave primaria. • NombreDeptno varchar(20) • Localidad varchar(20)	• Empno numérico clave primaria • NombreEmpno varchar (25) • Cargo varchar(15) • FechaIng date • Salario numérico • Comisión numérico • Deptno numerico clave ajena (a Departamentos).

Realiza las siguientes sentencias sobre las tablas **Departamentos y Empleados**.

Las sentencias serán las siguientes:

1. Módulo para la inserción de datos en ambas tablas
2. Módulo que permita actualizar datos en ambas tablas.
3. Módulo que permita eliminar registro de ambas tablas.
4. Mostrar el nombre y la localidad de todos los departamentos
5. Modificar el salario de los empleados que ganen entre de 1750 a 2250 .'
6. Mostrar los datos de los Empleados que pertenezcan a los departamentos cuya localidad sea Arucas y que tenga una comisión mayor de 20
7. Modifica la sentencias del apartado 5. Pidiendo al usuario los valores entro los que quieras modificar.
8. Modifica el apartado 6. Solicitando al usuario la localidad y la comisión.